

VulnPriority AI

Team ClaudeMaxers

Demo <https://vulnpriority-ai.onrender.com>

System Overview

VulnPriority AI is a fully agentic 9-stage pipeline that autonomously reviews CVEs from a Software Bill of Materials (SBOM) and VA/pentest report, assesses business and technical risk, and produces a **deterministic, auditable ranked patch calendar**. Every priority score uses a transparent formula from verified data sources (OSV.dev + NVD). Rankings are stable and reproducible across repeated runs.

9-Stage Pipeline

#	Stage	Type	Description
01	Input Parsing	Processing	Extract packages from SBOM, CVE IDs from VA report, merge to unified list
02	CVE Discovery	OSV Verified	OSV.dev ground-truth per package@version - 3-layer anti-hallucination
03	Deep CVE Research	NVD + LLM	NVD Primary CVSS + LLM: technical context, attack path, affected versions
04	PoC Generation	On-Demand	Gemini/Claude writes patch regression test (Python/JS/Bash) - user-triggered
05	Exploit Evaluation	Gemini/ Claude	Viability: High/Medium/Low - risk summary, remediation steps
06	Impact Analysis	Mathematical	Blast radius = ceil(services x AV x Scope) from dependency graph
07	Priority Scoring	Deterministic	CVSS x PoC x Blast / Complexity x Tier x Regulatory - auditable
08	Rationale Generation	LLM (Gemini/Claude)	Plain-English explanation per CVE: rank reason, fix guidance
09	Sched. & Assignment	Sprint Plan	Engineer skill matching + sprint capacity + maintenance windows

Blue = LLM-assisted Green = Mathematical/deterministic Purple = On-demand only

Priority Scoring Formula

Every CVE receives a **deterministic priority score** from verified data. Rankings are auditable and stable across repeated runs.

$$\text{Priority} = \text{CVSS} \times \text{PoC_mul} \times \text{BlastRadius} \div \text{PatchComplexity} \times \text{TierWeight} \times \text{RegFlag}$$

Factor	Source	Values
CVSS	NVD Primary score (never CNA fallback)	0.0 - 10.0
PoC Multiplier	Exploit viability (Claude)	x3.0 active x1.5 medium x1.0 none
Blast Radius	ceil(services x AV_factor x Scope_factor)	1 - 10
Patch Complexity	CVSS vector: AC + PR + UI fields	1 - 5
Tier Weight	System criticality tier	Critical x3.0 Important x2.0 Standard x1.0
Regulatory Flag	PCI / SOX / HIPAA / GDPR / FedRAMP	x2.0 if present x1.0 otherwise

Anti-Hallucination: 3-Layer CVE Validation

LLMs can hallucinate CVEs (e.g., assigning a 1999 Windows CVE to an npm package). Three sequential guards prevent this:

Layer	Guard	How It Works
1	OSV.dev ground truth	Returns only CVEs confirmed by OSV for the exact package@version
2	Year sanity check	npm CVEs must be >= 2010; rejects chronologically impossible CVEs
3	NVD CPE product match	CVE CPE must contain package name (word-boundary character check)

Technology Stack

Layer	Technology
Backend	Python 3.11 FastAPI 0.115 Server-Sent Events (SSE real-time streaming)
Frontend	React 18 (CDN, no build step) Highlight.js Babel standalone JSX
AI Research	Google Gemini gemini-2.0-flash via google-genai SDK
AI Evaluation	Gemini gemini-2.0-flash (primary), Claude claude-opus-4-5 (fallback) via respective SDKs
CVE Data	OSV.dev REST API + NVD REST API v2 (Primary CVSS, never CNA fallback)
Persistence	SQLite - scan history, team profiles, system configurations
Deployment	Render (Python web service - React frontend served by FastAPI)

Input Files (test_dataset_enterprise_2026/)

File	Section	Purpose
package.json	SBOM	All npm packages and exact versions to scan
va_report_q2_2026.txt	VA Report	Pentest report - CVE IDs extracted automatically
system_info.json	System	Name, tier (Critical/Important/Standard), regulatory scope
team_profiles.json	Team	Engineer skills, sprint hours, availability
maintenance_windows.json	Maintenance	Scheduled downtime windows for patch deployment
vendor_advisories.json	Connector	Vendor patch notices and affected packages
internal_docs.json	Connector	Internal threat intelligence and architecture context
dependency_graph.json	Connector	Service dependency edges for blast radius calc

Local Setup — Step by Step

Prerequisites: Python 3.11+, Gemini API key, Anthropic API key (optional), and optionally an NVD API key (nvd.nist.gov/developers) to remove the 5 req/30 s rate limit.

Step 1 — Clone

```
git clone https://github.com/Lord2729/WhiteHat.git
cd WhiteHat
```

Step 2 — Install dependencies

```
cd backend
pip install -r requirements.txt
```

Step 3 — Create backend/.env

```
GEMINI_API_KEY=Aiza...
ANTHROPIC_API_KEY=sk-ant-...
NVD_API_KEY=your-nvd-key # optional but recommended
```

Step 4 — Start backend

```
cd backend
uvicorn main:app --reload --port 8000
```

Step 5 — Open frontend

Go to <http://localhost:8000> — FastAPI serves React from ../frontend. No build step required.

Running a Scan - Quick Start

The repo ships with a complete enterprise simulation in `test_dataset_enterprise_2026/` — **NorthStar Payments**, a fictional PCI + GDPR + SOX processor with 7 downstream services and 5 engineers.

Step	Action
1	Open Configure page - upload package.json in the SBOM section
2	Upload va_report_q2_2026.txt in the VA Report section
3	Upload system_info.json, team_profiles.json, maintenance_windows.json
4	Upload vendor_advisories.json, internal_docs.json, dependency_graph.json
5	Click Run Agentic Analysis - pipeline streams live progress (~90 s with both API keys)
6	Results appear automatically - explore five tabs: Vulns / Calendar / Kanban / PoC / Dashboard

Results Views

Tab	Contents
Ranked Vulns	CVEs by priority score with expandable rows (CVSS, blast radius, PoC viability, assigned engineer)
Patch Calendar	Patches scheduled within maintenance windows, assigned engineers, dates
Kanban Board	Sprint tickets: Backlog / In Progress / Complete
PoC Exploits	On-demand patch regression test generator per CVE - Python, JavaScript, or Bash
Dashboard	Risk delta, effort distribution, engineer workload, scan summary metrics

Key API Endpoints

Method	Endpoint	Description
GET	/health	Service health check - confirms backend is running
POST	/api/analyze	Run full 9-stage pipeline - SSE stream response
POST	/api/generate-exploit	On-demand PoC / patch regression test for a single CVE
POST	/api/upload-report	Upload VA report text - CVE IDs extracted automatically
GET	/api/config/load	Load saved system configuration from SQLite
POST	/api/config/save	Persist system configuration to SQLite
GET	/api/scans	List all saved scan results
GET	/api/scans/{id}	Load a specific saved scan result
GET	/api/team-profiles	Get team engineer profiles
GET	/api/sample-input	Load built-in demo dataset - no file upload needed

Cloud Deployment (Render)

Repo includes `render.yaml` for one-click deployment. FastAPI serves the React frontend no separate static host needed.

#	Action	Value / Command
1	Fork repo	github.com/Lord2729/WhiteHat → Fork
2	New Web Service	dashboard.render.com → New → Web Service → Connect repo
3	Root Directory	backend
4	Build Command	<code>pip install -r requirements.txt</code>
5	Start Command	<code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code>
6	Python Version	backend/.python-version must contain: 3.11.9
7	Env Variables	GEMINI_API_KEY, ANTHROPIC_API_KEY, NVD_API_KEY
8	Deploy	Click Deploy Web Service (~2 min build)

Important: Without `backend/.python-version = 3.11.9`, Render defaults to Python 3.14 causing pydantic-core build failure (no pre-built wheel). File is pre-committed in the repo.

Troubleshooting

Symptom	Cause	Fix
Backend Offline banner	Render free-tier cold start (~30 s)	Hard-refresh (Ctrl+Shift+R) after ~30 seconds
NVD PENDING scores	NVD rate-limit (no API key)	Add NVD_API_KEY env var (free at nvd.nist.gov)
PoC returns empty code	Gemini safety filter	Mitigated - prompt reframed as patch regression test
Pipeline NameError	Stale .pyc bytecode	Restart uvicorn - Python recompiles from source

Pipeline Architecture Diagram

The diagram illustrates the complete 9-stage pipeline from SBOM + VA report inputs to the ranked patch calendar.

Blue = LLM-assisted, Green = Mathematical, Purple = On-demand.

